Combinatorial Optimization Project 2: Linear Programming - Simplex

Requirements before starting:

1) you must work in teams of 2 and no more than 2 (pick a team from any section feel free to find a team using a new thread on the discussion broad if needed, no requirements on same section-teams) unless you have a compelling reason in which case please email me
2) This project requires that you have already completed all the formative assessments above.
3) Include the links to your video and to your code on Page 1 of your project
4) Include the team member names on Page 1 at the top along with the Project # and Date
5) Name your file starting with Course then semester, then Project Number, then both Last names of the team members, separated by underscores, such as CSE620_Fall26_Project2_Name1_Name2

Overall goal:

In this project, you will use the Simplex on a series of increasingly larger LP problems. You will study the solution, the time to find the solution, and feasibility for problems of increasing size.

Corresponding Unit Learning Objectives for this project:

- Use the Simplex algorithm to solve LP problems.

Aligned to Course Learning Objective:

1. Demonstrate theoretical and practical knowledge of classical optimization methods.
3. Write an application or adapt an existing numerical or combinatorial optimization code or tool for implementing optimization algorithms, such as the Gradient Descent, Simplex algorithm, simulated annealing, genetic algorithms, genetic niching, multi-objective optimization, etc.

Specific project tasks that will help you achieve the above learning objectives:

To achieve the above objectives, the project requires you to:

- Correctly implement or apply Simplex (*see links below for possible implementations*).
- Set up LP problems of increasing size by
    - Increasing the number of variables *n* in the problem
    - Increasing the number of constraints *m* in the problem
- Analyze the impact of *m* and *n* on the system time and number of pivots needed for solving the LP and the solution that you found for each of the m,n combinations.

Implementation/Code:

Find and download any source code or executable or existing optimization toolbox containing the Simplex algorithm for the LP problem.

Examples:

Python SciPy library for Linear Programming (*recommended*):
https://docs.scipy.org/doc/scipy/reference/generated/scipy.optimize.linprog.html

Google developer code for Combinatorial Optimization:

https://developers.google.com/optimization/introduction

Matlab Optimization ToolBox: https://www.mathworks.com/products/optimization.html

What to Implement / Do:

A. Run your code (or tool) on the example LP problem instance from the lecture on Simplex (n=4, m=2)

B. Next try to increase the number of variables (n) and increase the number of constraints (m), thus:

- Fixing m=2: Rerun the code after increasing n from 4 to 10 to 50. (in increments of 10) and n

- For each of the n values above: Rerun the code after increasing m from 2 to 6 to 10 to 14. (in increments of 4, just create additional constraints)

C. Record the **system time and number of pivots** to obtain each solution, versus the problem size.
D. Analyze your results by discussing what you observe as the problem size and constraints vary, etc.

Expected Phenomena to Observe:

- Some problems will lead to solutions quickly while others will not have solutions. Why?
- The number of variables and number of constraints have an impact on the number of vertices in the simplex: How does this impact the time and number of pivots to solve the LP?
- Problems with more variables/constraints may take longer to solve
- Some LPs may be infeasible: identify why
- Removing constraints can lead to unbounded solutions.
- Multiple optimal solutions (degeneracy) may appear.
- Sparsity: in many real problems, most decision variables are zero in the optimal solution. Why? What does this mean in real life applications?

Deliverables: (report, code and video):

Report: Return your answer as a well structured and organized report of no more than 8 pages + appendix for plots and tables IF too many, with sections for each of the following:

- **Introduction Section:** stating the objective of the project and the optimization problem instances using **equations.**

- **Methods Section**:
  - Describing the Simplex algorithm you have used in **pseudocode and equations.**
  - Describing the optimization code that you used, including its source code and simple examples of how to use it If you did not use the above notebook (and on which platform).

- **Results Section**
  - **Tabulate** and **plot** for each problem the number of variables, constraints, the system time to find a solution, number of pivots to the solution, the cost value of the solutions obtained.

- **Experimental Analysis Section**: An analysis of all your above results.
  - What do you observe as the problem size and constraints vary, etc?

- **Conclusion Section:** describe your overall findings based on the experiments and what you learned and could improve.

- **Appendix Section** (includes code, and any excess tabular or graph results that do not fit the maximum number of pages)

Code: file or link is acceptable.

Video: Additionally, record a brief ~5 minute video sharing your screen and following this format:

- Demo your notebook/code step-by-step, outlining your methodology and experiments (~2 minutes)
- Summarize/discuss your findings, things you thought were interesting, etc. (~2 minutes)
- Specify the most challenging/rewarding aspects of the project, and why (~1 minute)
- **It is preferred if you can provide a link to your recording rather than the file itself. A couple of examples you can try:**
    - Microsoft Sharepoint (please make the recording visible through the link you submit)
    - Google Drive (upload Mp4 to drive → share file by link)
    - YouTube (unlisted video with link)

Please submit your assignment as a compressed file containing the above (minus the video) named like **[Group member 1 last name]_[Group member 2 last name]_Assignment1.zip**, where last names are arranged alphabetically.

Grading Rubric (100 pts):

1. **Structure and completeness of the Report** with Adherence to page limit, all above required sections and all required Content as described above, and including: (80 points) divided into:
    - **Methods & thorough organized reporting of results** (**tables & figures**) (20 points)
    - **Quality of experiments** (20 pts)
        - **Comparison** (number of pivots, time to solve, final cost, etc)
    - **Analysis depth & insight** (20 pts)
        - **Reflections**: When is Simplex fast or slow? When does it fail to yield a solution? … etc. **See phenomena to observe section above and point to your experimental results as evidence** when making conclusions or observations.
2. **Code organization & readability** (10 pts)
3. **Video** - adherence to time limit and comprehensive explanation that shows you master the project methods and findings (10)